

The AAA Pattern



A **common structure** used to **organize test method** code clearly

 by **Rodrigo Martins Pagliares**

Last update: 11/26/2025

Arrange, Act, Assert



Arrange

Prepare objects, **inputs**, and dependencies needed for the test

Act

Invoke the **method under test** using the **arranged inputs**

Assert

Verify that the **result matches** the **expected behavior**

Keeps the test readable by dividing responsibilities cleanly

Example With Integer Division



- **Arrange:** Create a Calculator instance and prepare integers like dividend/divisor
- **Act:** Call `integerDivision(dividend, divisor)` and capture the actual result
- **Assert:** Compare the actual result with the expected value using assertions
- Clear structure makes test intention obvious at a glance

Example With Integer Division

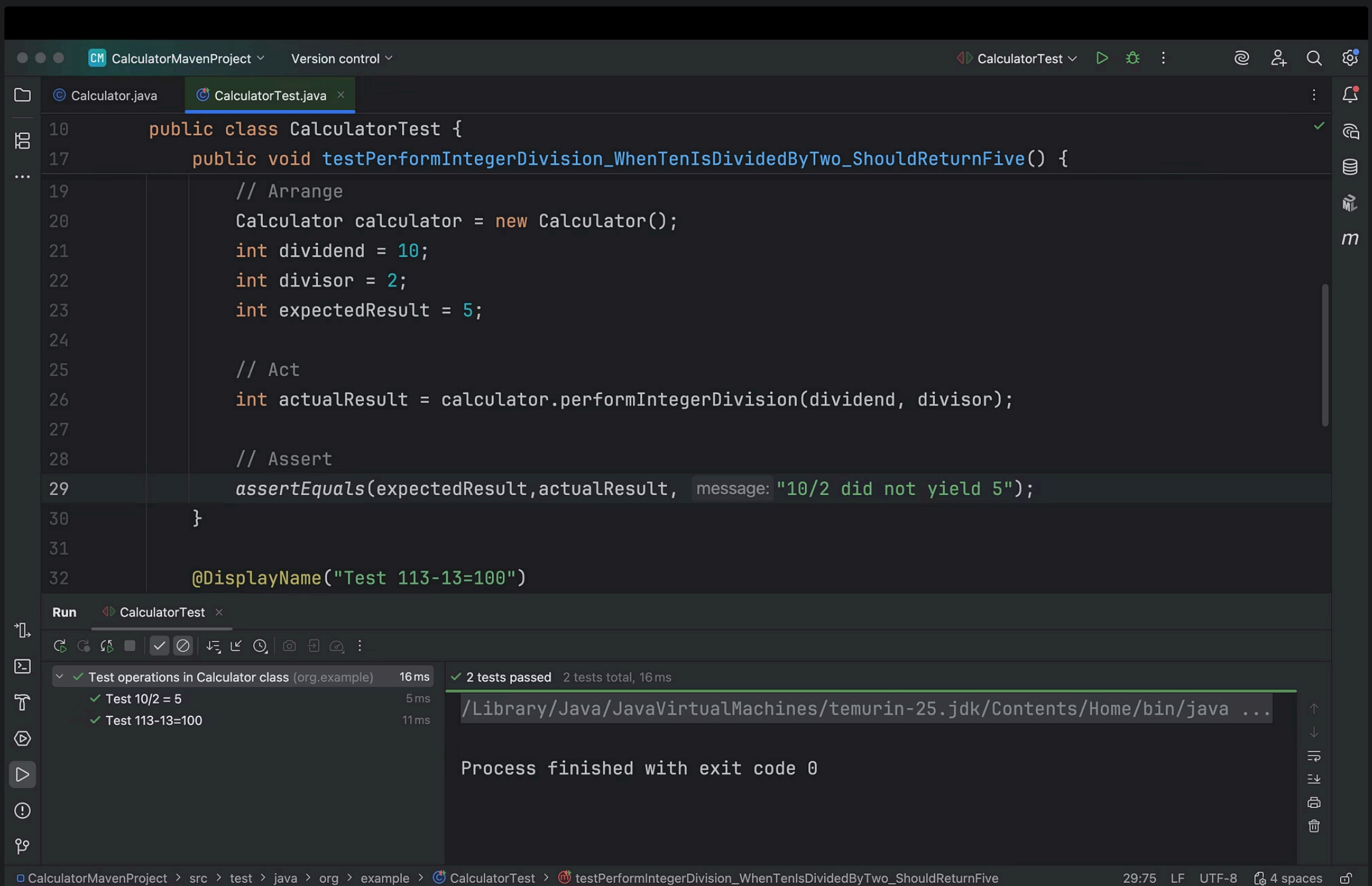
```
Calculator.java  CalculatorTest.java x
10      public class CalculatorTest {
14
15          @DisplayName("Test 10/2 = 5")
16          @Test
17      public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {
18          // Arrange
19
20          // Act
21
22          // Assert
23          Calculator calculator = new Calculator();
24          int result = calculator.performIntegerDivision(dividend: 10, divisor: 2);
25          assertEquals(expected: 5, result, message: "10/2 did not yield 5");
26      }
27
28          @DisplayName("Test 113-13=100")
```

- **Arrange:** Create a `Calculator` instance and prepare **integers** like `dividend/divisor`
- **Act:** Call `integerDivision(dividend, divisor)` and capture the actual result
- **Assert:** Compare the **actual result** with the **expected value** using assertions
- **Clear structure** makes **test intention** obvious at a glance

First attempt at refactoring the code to adhere to the AAA pattern (Success!)

```
Calculator.java CalculatorTest.java x
10 public class CalculatorTest {
11     // Template for naming test methods:
12     // test<method>_<condition>_<expectedResult>
13
14     @DisplayName("Test 10/2 = 5")
15     @Test
16     public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {
17         // Arrange
18         Calculator calculator = new Calculator();
19         // Act
20         int result = calculator.performIntegerDivision(dividend: 10, divisor: 2);
21         // Assert
22         assertEquals(expected: 5, result, message: "10/2 did not yield 5");
23     }
24
25     @DisplayName("Test 113-13=100")
26 }
```

Second attempt at refactoring the code to adhere to the AAA pattern (Success!)



The screenshot displays an IDE window for a project named 'CalculatorMavenProject'. The active file is 'CalculatorTest.java', which contains the following code:

```
10 public class CalculatorTest {
17     public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {
19         // Arrange
20         Calculator calculator = new Calculator();
21         int dividend = 10;
22         int divisor = 2;
23         int expectedResult = 5;
24
25         // Act
26         int actualResult = calculator.performIntegerDivision(dividend, divisor);
27
28         // Assert
29         assertEquals(expectedResult, actualResult, message: "10/2 did not yield 5");
30     }
31
32     @DisplayName("Test 113-13=100")
```

Below the code editor, the 'Run' tab shows the execution results for 'CalculatorTest'. The test suite 'Test operations in Calculator class (org.example)' passed with a total time of 16 ms. The individual tests and their durations are:

- ✓ Test 10/2 = 5 (5 ms)
- ✓ Test 113-13=100 (11 ms)

The console output shows the command: `/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...` and the message: `Process finished with exit code 0`.

The bottom status bar indicates the current file path: `CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive`, along with encoding settings: `29:75 LF UTF-8 4 spaces`.

Alternative Naming: **Given–When–Then**



Given

Setup context and preconditions



When

Execute the behavior being tested



Then

Validate the outcome or side effects

Some teams prefer **Given–When–Then** instead of **AAA**

Provides a **natural-language feel** similar to **BDD practices**

Gherkin Syntax: Given, When, Then

```
Calculator.java CalculatorTest.java x
10 public class CalculatorTest {
16     @Test
17     public void testPerformIntegerDivision_WhenTenIsDividedByTwo_ShouldReturnFive() {
18
19         // Arrange (Given)
20         Calculator calculator = new Calculator();
21         int dividend = 10;
22         int divisor = 2;
23         int expectedResult = 5;
24
25         // Act (When)
26         int actualResult = calculator.performIntegerDivision(dividend, divisor);
27
28         // Assert (Then)
29         assertEquals(expectedResult, actualResult, message: "10/2 did not yield 5");
30     }
```

Arrange, Act, Assert (AAA) is also known as **Given, When, Then** (Gherkin syntax)

Conclusion

- **AAA** (or **Given–When–Then**) improves **readability** and **structure**
- Helps maintain **clarity** even as **test suites** grow large
- Works well alongside conventions like descriptive test names and `@DisplayName`
- Reinforces good **testing habits** that scale across **projects** and **teams**



Open Floor Discussion



**Do you prefer AAA
or Given–When–
Then for your test
structure?**

**Would adopting a
consistent pattern
help your students
or teammates?**

**How might this
structure
integrate with
your current
testing style?**